UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería

Escuela de Ingeniería en Ciencias y Sistemas

Introducción a la Programación y Computación 1

TAREA 4

Programación Orientada a Objetos, Validaciones e Interfaz Gráfica con Java Swing

Laboratorio IPC 1 • Segundo Semestre 2026

Curso:	0770 - Introducción a la Programación y Computación 1
Actividad:	Tarea 4 - Gestión de Cursos, Tareas y Conversor de Temperatura
Estudiante:	[Nombre del Estudiante]
Carné:	[Carné del Estudiante]
Tutor / Auxiliar:	[Nombre del Tutor] - Sección [A/B/C/D]
Fecha de Entrega:	Octubre de 2026

Guatemala, Centroamérica • Universidad de San Carlos de Guatemala (USAC)

1. Sección de Investigación Teórica (Saber)

El desarrollo de software profesional bajo el paradigma orientado a objetos demanda una comprensión cabal de sus fundamentos y de las herramientas gráficas provistas por la plataforma Java. A continuación, se desarrollan con rigor conceptual los seis temas requeridos en la guía académica.

1.1 Tema 1: Programación Orientada a Objetos: Clase, Objeto, Atributo y Método

La **Programación Orientada a Objetos (POO)** es un paradigma de programación que organiza el diseño de software alrededor de datos u objetos, en lugar de funciones y lógica procedimental. Un objeto se define como un campo de datos que tiene atributos y comportamientos únicos.

- **Clase:** Es una plantilla, plano o modelo conceptual que define la estructura y el comportamiento común de un conjunto de objetos. No ocupa memoria para instancias de datos en sí misma, sino que especifica qué variables y métodos tendrán los objetos construidos a partir de ella (ejemplo: la clase `Curso`).
- **Objeto:** Es una entidad concreta e individualizada en tiempo de ejecución construida a partir de una clase mediante el operador `new`. Posee un estado (valores de sus atributos), comportamiento (métodos) e identidad única en memoria (ejemplo: la instancia de `Curso` con código "0770").
- **Atributo:** Variable declarada dentro del cuerpo de la clase que almacena el estado y las propiedades específicas de cada objeto (por ejemplo, `private String codigo;` o `private String nombre;`).
- **Método:** Subrutina o función definida dentro de una clase que ejecuta una tarea o cálculo y que manipula o consulta el estado de los atributos del objeto (por ejemplo, `agregarTarea(...)` o `getCodigo()`).

1.2 Tema 2: Los Cuatro Pilares de la Programación Orientada a Objetos

Pilar	Definición y Fundamento Conceptual	Implementación en Java
Encapsulamiento	Consiste en ocultar los detalles internos y el estado de un objeto respecto a componentes externos. Protege la integridad de los datos evitando mutaciones indebidas y desacopla la lógica interna de su uso público.	Uso de visibilidad privada (`private`) en los campos junto a métodos públicos (`public getters / setters`).
Abstracción	Habilidad de extraer y modelar únicamente las características y comportamientos esenciales de una entidad para el contexto del problema, ignorando los detalles accesorios o irrelevantes.	Creación de las clases `Curso` y `TareaAcademica` conteniendo exclusivamente los campos requeridos para el control escolar.
Herencia	Mecanismo que permite a una nueva clase (subclase) adquirir automáticamente atributos y comportamientos de una clase existente (superclase), promoviendo la reutilización y extensibilidad.	Palabra reservada `extends` , utilizada al construir `VentanaPrincipal extends JFrame` para heredar la funcionalidad gráfica de ventana.
Polimorfismo	Propiedad que permite a objetos de distintas clases responder a una misma invocación de método de forma adaptada a su naturaleza. Incluye polimorfismo dinámico (sobreescritura) y estático (sobrecarga).	Sobreescritura del método `@Override public String toString()` y la implementación polimórfica de eventos en `actionPerformed` .

1.3 Tema 3: Diseño de Clases para Representar Cursos y Tareas Académicas

En el análisis del dominio académico, se identifican dos conceptos centrales: el **Curso** y la **Tarea Académica**.

> **Relación Estructural: Composición / Agregación 1 a N**
>
> Un `Curso` es una entidad principal que agrupa actividades curriculares. En nuestro diseño POO, cada objeto `Curso` almacena su identificador (`codigo`), título (`nombre`) y docente (`tutor`), así como una colección dinámica de tareas (`List<TareaAcademica>`).
>
> Por su parte, cada `TareaAcademica` posee propiedades puntuales (`titulo` , `descripcion` , `fechaEntrega`) y una referencia a su curso contenedor. De esta forma, las tareas no flotan de manera huérfana en la memoria, sino que están estrictamente vinculadas a la asignatura correspondiente, posibilitando consultas jerárquicas y consistencia relacional.

1.4 Tema 4: Componentes Fundamentales de Java Swing

Java Swing provee un conjunto completo de componentes visuales ligeros ("lightweight") e independientes del sistema operativo para edificar interfaces de usuario:

Componente	Descripción y Responsabilidad	Aplicación en la Práctica
`JFrame`	Ventana principal de primer nivel con decoraciones del sistema operativo (título, botones minimizar/cerrar). Contiene el panel raíz de la aplicación.	Marco contenedor de toda la aplicación (`VentanaPrincipal`).
`JPanel`	Contenedor intermedio no decorado utilizado para seccionar y ordenar la interfaz mediante administradores de diseño (`LayoutManager`).	Estructuración en módulos con `BorderFactory.createTitledBorder` .
`JLabel`	Componente estático para mostrar texto o imágenes descriptivas que orientan al usuario sobre el contenido o indican resultados de cálculos.	Etiquetas descriptivas de entradas y visualización del resultado de conversión.
`JTextField`	Control editable de una sola línea diseñado para la captura de texto o valores numéricos introducidos por teclado.	Campos de entrada para código, nombre, tutor, descripción y grados Celsius.
`JButton`	Botón interactivo capaz de detonar eventos de usuario al recibir clics del ratón o atajos de teclado.	Botones de registro de cursos, adición de tareas, conversión y reportes.
`JTextArea`	Área multilínea de texto. Admite texto plano enriquecido mediante fuentes monoespaciadas y barras de desplazamiento con `JScrollPane` .	Consola central de reporte donde se listan los cursos con sus tareas.
`JOptionPane`	Clase de utilidad para crear fácilmente cuadros de diálogo modales que notifican información, advierten riesgos o alertan sobre errores de captura.	Manejo de mensajes emergentes para validación de campos y confirmaciones.

1.5 Tema 5: Manejo de Eventos en Java: ActionListener y Respuesta a Botones

El manejo de eventos en Swing implementa el **Modelo de Delegación de Eventos** (Event Delegation Model). Bajo este esquema:

1. La interacción física (por ejemplo, hacer clic sobre un `JButton`) genera un evento denominado `ActionEvent` .
2. El componente emisor delega el procesamiento del evento a uno o más escuchas registrados mediante el método `addActionListener(ActionListener l)` .
3. La interfaz `ActionListener` contiene el método abstracto `void actionPerformed(ActionEvent e)` , donde se redacta el código que se debe ejecutar en respuesta al clic.

En la aplicación desarrollada, se implementan clases internas anónimas para conectar limpiamente cada botón con su método controlador de negocio correspondiente (`registrarCurso()` , `agregarTarea()` , etc.), garantizando una separación clara entre la capa de interfaz y la lógica de negocio.

1.6 Tema 6: Validación de Entradas Numéricas y Textuales

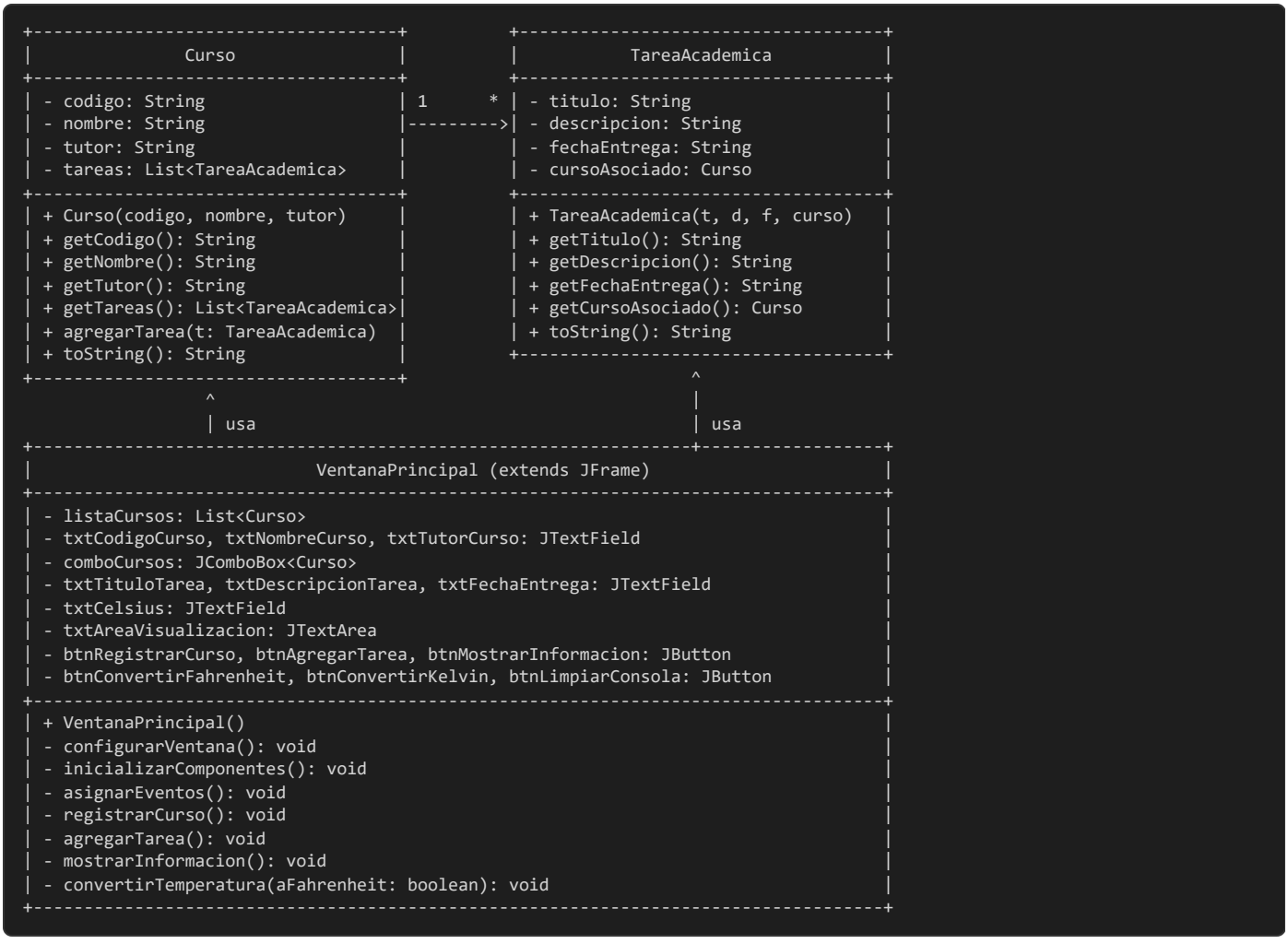
La validación preventiva de datos es una práctica obligatoria para asegurar la robustez de las aplicaciones:

- **Validación Textual:** Al recuperar el valor de un campo con `getText()`, se aplica el método `trim()` para descartar espacios en blanco accidentales al inicio y al final. Luego se verifica con `isEmpty()` si el usuario omitió ingresar información. Si el campo está vacío, se interrumpe la operación y se despliega un `JOptionPane.WARNING_MESSAGE`.
- **Validación Numérica:** En campos donde se esperan valores cuantitativos (como la temperatura en Celsius), se intenta transformar la cadena mediante `Double.parseDouble(texto)`. Esta operación se envuelve obligatoriamente en un bloque `try-catch(NumberFormatException ex)`. Si la conversión falla debido a que el usuario ingresó letras o símbolos, la excepción es capturada ordenadamente y se le notifica con un `JOptionPane.ERROR_MESSAGE`, impidiendo que el programa se cierre abruptamente.

2. Sección Práctica - Diseño y Arquitectura de la Solución (Saber Hacer)

2.1 Diseño de Clases y Relaciones UML

De acuerdo con las directrices de la cátedra, la aplicación fue construida **únicamente mediante código Java puro**, prescindiendo por completo de editores visuales de arrastrar y soltar (GUI Builders). Esto permite un dominio estricto del árbol de componentes de Swing y garantiza un código limpio, estructurado y fácil de mantener.



2.2 Fórmulas Matemáticas del Módulo de Conversión

El módulo de conversión aplica rigurosamente las ecuaciones físicas estándar de conversión térmica a partir de grados Celsius (°C):

> **1. Fórmula de Conversión a Grados Fahrenheit:**
> °F = (°C × 9/5) + 32 = (°C × 1.8) + 32
> *Ejemplo del enunciado:* Si ingresa **25 °C** → (25 × 1.8) + 32 = 45 + 32 = **77 °F**.

2. Fórmula de Conversión a Kelvin:
K = °C + 273.15
Ejemplo del enunciado: Si ingresa **25 °C** → 25 + 273.15 = **298.15 K**.

3. Código Fuente Completo y Documentado

3.1 Clase de Entidad: Curso.java

```
package tarea4.modelo;

import java.util.ArrayList;
import java.util.List;

/**
 * Clase que representa un Curso académico.
 * Aplica los principios de POO y encapsulamiento.
 */
public class Curso {
    // Atributos privados (Encapsulamiento)
    private String codigo;
    private String nombre;
    private String tutor;
    private List<TareaAcademica> tareas;

    /**
     * Constructor para inicializar los datos del curso.
     * @param codigo Código identificador del curso (ej. 0770)
     * @param nombre Nombre de la asignatura (ej. IPC1)
     * @param tutor Nombre del tutor responsable
     */
    public Curso(String codigo, String nombre, String tutor) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.tutor = tutor;
        this.tareas = new ArrayList<>();
    }

    // Métodos Getters y Setters
    public String getCodigo() {
        return codigo;
    }

    public void setCodigo(String codigo) {
        this.codigo = codigo;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getTutor() {
        return tutor;
    }

    public void setTutor(String tutor) {
        this.tutor = tutor;
    }

    public List<TareaAcademica> getTareas() {
        return tareas;
    }

    /**
     * Agrega una tarea asociada a este curso.
     * @param tarea Tarea académica a asociar
     */
    public void agregarTarea(TareaAcademica tarea) {
```

```
        if (tarea != null) {
            this.tareas.add(tarea);
        }
    }

    /**
     * Representación textual amigable para componentes como JComboBox.
     */
    @Override
    public String toString() {
        return codigo + " - " + nombre;
    }
}
```

3.2 Clase de Entidad: TareaAcademica.java

```
package tarea4.modelo;

/**
 * Clase que representa una Tarea Académica asociada a un curso.
 * Aplica los principios de POO y encapsulamiento.
 */
public class TareaAcademica {
    // Atributos privados
    private String titulo;
    private String descripcion;
    private String fechaEntrega;
    private Curso cursoAsociado;

    /**
     * Constructor de la tarea académica.
     * @param titulo Título de la tarea
     * @param descripcion Detalle o descripción de la tarea
     * @param fechaEntrega Fecha límite de entrega
     * @param cursoAsociado Referencia al curso al que pertenece
     */
    public TareaAcademica(String titulo, String descripcion, String fechaEntrega, Curso cursoAsociado) {
        this.titulo = titulo;
        this.descripcion = descripcion;
        this.fechaEntrega = fechaEntrega;
        this.cursoAsociado = cursoAsociado;
    }

    // Métodos Getters y Setters
    public String getTitulo() {
        return titulo;
    }

    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public String getDescripcion() {
        return descripcion;
    }

    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }

    public String getFechaEntrega() {
        return fechaEntrega;
    }

    public void setFechaEntrega(String fechaEntrega) {
        this.fechaEntrega = fechaEntrega;
    }

    public Curso getCursoAsociado() {
        return cursoAsociado;
```

```
    }

    public void setCursoAsociado(Curso cursoAsociado) {
        this.cursoAsociado = cursoAsociado;
    }

    @Override
    public String toString() {
        return "Tarea: " + titulo + " | Entrega: " + fechaEntrega;
    }
}
```

3.3 Clase Gráfica Principal: VentanaPrincipal.java

```
package tarea4.vista;

import tarea4.modelo.Curso;
import tarea4.modelo.TareaAcademica;

import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.TitledBorder;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.text.DecimalFormat;
import java.util.ArrayList;
import java.util.List;

/**
 * Ventana Principal de la aplicación desarrollada con Java Swing.
 * Programada 100% mediante código (sin herramientas de arrastrar y soltar).
 * Integra los módulos de Cursos, Tareas, Visualización y Conversor de Temperatura.
 */
public class VentanaPrincipal extends JFrame {

    // Lista principal para almacenar los cursos registrados
    private List<Curso> listaCursos;

    // Componentes del Módulo de Cursos
    private JTextField txtCodigoCurso;
    private JTextField txtNombreCurso;
    private JTextField txtTutorCurso;
    private JButton btnRegistrarCurso;

    // Componentes del Módulo de Tareas
    private JComboBox<Curso> comboCursos;
    private JTextField txtTituloTarea;
    private JTextField txtDescripcionTarea;
    private JTextField txtFechaEntrega;
    private JButton btnAgregarTarea;

    // Componentes del Módulo de Conversión
    private JTextField txtCelsius;
    private JButton btnConvertirFahrenheit;
    private JButton btnConvertirKelvin;
    private JLabel lblResultadoConversion;

    // Componentes del Módulo de Visualización
    private JTextArea txtAreaVisualizacion;
    private JButton btnMostrarInformacion;
    private JButton btnLimpiarConsola;

    // Formateador para números decimales
    private DecimalFormat df = new DecimalFormat("#.##");

    /**
     * Constructor principal de la ventana.
     */
    public VentanaPrincipal() {
        super("Control Académico y Conversor de Temperatura - Tarea 4 IPC1");
        this.listaCursos = new ArrayList<>();

        configurarVentana();
        inicializarComponentes();
        asignarEventos();
    }

    /**
     * Configuración básica del JFrame.
     */
    private void configurarVentana() {
```

```
        this.setSize(950, 720);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        this.setLocationRelativeTo(null); // Centra la ventana en pantalla
        this.setLayout(new BorderLayout(10, 10));
    }

    /**
     * Inicializa y organiza los componentes gráficos por código.
     */
    private void inicializarComponentes() {
        // Panel contenedor principal con padding
        JPanel panelContenedor = new JPanel(new GridLayout(1, 2, 12, 12));
        panelContenedor.setBorder(new EmptyBorder(12, 12, 12, 12));

        // ========================================
        // COLUMNA IZQUIERDA: FORMULARIOS Y CONVERSIÓN
        // ========================================
        JPanel panelIzquierdo = new JPanel();
        panelIzquierdo.setLayout(new BoxLayout(panelIzquierdo, BoxLayout.Y_AXIS));

        // 1. Panel de Cursos
        JPanel panelCursos = new JPanel(new GridBagLayout());
        panelCursos.setBorder(BorderFactory.createTitledBorder(
                BorderFactory.createEtchedBorder(), "1. Registro de Cursos",
                TitledBorder.LEFT, TitledBorder.TOP, new Font("Segoe UI", Font.BOLD, 13), new Color(25, 55, 109)));

        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(4, 5, 4, 5);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        // Código
        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0.3;
        panelCursos.add(new JLabel("Código del Curso:"), gbc);
        gbc.gridx = 1; gbc.gridy = 0; gbc.weightx = 0.7;
        txtCodigoCurso = new JTextField(15);
        panelCursos.add(txtCodigoCurso, gbc);

        // Nombre
        gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0.3;
        panelCursos.add(new JLabel("Nombre del Curso:"), gbc);
        gbc.gridx = 1; gbc.gridy = 1; gbc.weightx = 0.7;
        txtNombreCurso = new JTextField(15);
        panelCursos.add(txtNombreCurso, gbc);

        // Tutor
        gbc.gridx = 0; gbc.gridy = 2; gbc.weightx = 0.3;
        panelCursos.add(new JLabel("Tutor Responsable:"), gbc);
        gbc.gridx = 1; gbc.gridy = 2; gbc.weightx = 0.7;
        txtTutorCurso = new JTextField(15);
        panelCursos.add(txtTutorCurso, gbc);

        // Botón Registrar Curso
        gbc.gridx = 0; gbc.gridy = 3; gbc.gridwidth = 2;
        btnRegistrarCurso = new JButton("Registrar Curso");
        btnRegistrarCurso.setBackground(new Color(230, 240, 255));
        btnRegistrarCurso.setFont(new Font("Segoe UI", Font.BOLD, 12));
        panelCursos.add(btnRegistrarCurso, gbc);

        panelIzquierdo.add(panelCursos);
        panelIzquierdo.add(Box.createVerticalStrut(10));

        // 2. Panel de Tareas
        JPanel panelTareas = new JPanel(new GridBagLayout());
        panelTareas.setBorder(BorderFactory.createTitledBorder(
                BorderFactory.createEtchedBorder(), "2. Registro de Tareas Académicas",
                TitledBorder.LEFT, TitledBorder.TOP, new Font("Segoe UI", Font.BOLD, 13), new Color(25, 55, 109)));

        gbc = new GridBagConstraints();
        gbc.insets = new Insets(4, 5, 4, 5);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        // Selección de Curso
        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0.3;
```

```
        panelTareas.add(new JLabel("Asociar a Curso:"), gbc);
        gbc.gridx = 1; gbc.gridy = 0; gbc.weightx = 0.7;
        comboCursos = new JComboBox<>();
        panelTareas.add(comboCursos, gbc);

        // Título de la Tarea
        gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0.3;
        panelTareas.add(new JLabel("Título de Tarea:"), gbc);
        gbc.gridx = 1; gbc.gridy = 1; gbc.weightx = 0.7;
        txtTituloTarea = new JTextField(15);
        panelTareas.add(txtTituloTarea, gbc);

        // Descripción
        gbc.gridx = 0; gbc.gridy = 2; gbc.weightx = 0.3;
        panelTareas.add(new JLabel("Descripción:"), gbc);
        gbc.gridx = 1; gbc.gridy = 2; gbc.weightx = 0.7;
        txtDescripcionTarea = new JTextField(15);
        panelTareas.add(txtDescripcionTarea, gbc);

        // Fecha de Entrega
        gbc.gridx = 0; gbc.gridy = 3; gbc.weightx = 0.3;
        panelTareas.add(new JLabel("Fecha de Entrega:"), gbc);
        gbc.gridx = 1; gbc.gridy = 3; gbc.weightx = 0.7;
        txtFechaEntrega = new JTextField(15);
        panelTareas.add(txtFechaEntrega, gbc);

        // Botón Agregar Tarea
        gbc.gridx = 0; gbc.gridy = 4; gbc.gridwidth = 2;
        btnAgregarTarea = new JButton("Agregar Tarea");
        btnAgregarTarea.setBackground(new Color(230, 240, 255));
        btnAgregarTarea.setFont(new Font("Segoe UI", Font.BOLD, 12));
        panelTareas.add(btnAgregarTarea, gbc);

        panelIzquierdo.add(panelTareas);
        panelIzquierdo.add(Box.createVerticalStrut(10));

        // 3. Panel de Conversión de Temperatura
        JPanel panelConversion = new JPanel(new GridBagLayout());
        panelConversion.setBorder(BorderFactory.createTitledBorder(
                BorderFactory.createEtchedBorder(), "3. Módulo de Conversión de Temperatura",
                TitledBorder.LEFT, TitledBorder.TOP, new Font("Segoe UI", Font.BOLD, 13), new Color(25, 55, 109)));

        gbc = new GridBagConstraints();
        gbc.insets = new Insets(5, 5, 5, 5);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        // Entrada Celsius
        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0.4;
        panelConversion.add(new JLabel("Grados Celsius (°C):"), gbc);
        gbc.gridx = 1; gbc.gridy = 0; gbc.weightx = 0.6;
        txtCelsius = new JTextField(10);
        panelConversion.add(txtCelsius, gbc);

        // Botones de conversión en fila horizontal
        JPanel panelBotonesTemp = new JPanel(new FlowLayout(FlowLayout.CENTER, 5, 0));
        btnConvertirFahrenheit = new JButton("Convertir a Fahrenheit");
        btnConvertirKelvin = new JButton("Convertir a Kelvin");
        panelBotonesTemp.add(btnConvertirFahrenheit);
        panelBotonesTemp.add(btnConvertirKelvin);

        gbc.gridx = 0; gbc.gridy = 1; gbc.gridwidth = 2;
        panelConversion.add(panelBotonesTemp, gbc);

        // Etiqueta de resultado
        lblResultadoConversion = new JLabel("Resultado: Esperando conversión...", SwingConstants.CENTER);
        lblResultadoConversion.setFont(new Font("Segoe UI", Font.BOLD, 13));
        lblResultadoConversion.setForeground(new Color(0, 102, 51));
        gbc.gridx = 0; gbc.gridy = 2; gbc.gridwidth = 2;
        panelConversion.add(lblResultadoConversion, gbc);

        panelIzquierdo.add(panelConversion);

        // ========================================
```

```
        // COLUMNA DERECHA: VISUALIZACIÓN Y REPORTES
        // ========================================
        JPanel panelDerecho = new JPanel(new BorderLayout(8, 8));
        panelDerecho.setBorder(BorderFactory.createTitledBorder(
                BorderFactory.createEtchedBorder(), "4. Visualización de Cursos y Tareas Registradas",
                TitledBorder.LEFT, TitledBorder.TOP, new Font("Segoe UI", Font.BOLD, 13), new Color(25, 55, 109)));

        // Área de texto con scroll
        txtAreaVisualizacion = new JTextArea();
        txtAreaVisualizacion.setEditable(false);
        txtAreaVisualizacion.setFont(new Font("Consolas", Font.PLAIN, 13));
        txtAreaVisualizacion.setMargin(new Insets(8, 8, 8, 8));
        JScrollPane scrollVisualizacion = new JScrollPane(txtAreaVisualizacion);
        panelDerecho.add(scrollVisualizacion, BorderLayout.CENTER);

        // Panel de botones de visualización
        JPanel panelBotonesVisualizacion = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 5));
        btnLimpiarConsola = new JButton("Limpiar Pantalla");
        btnMostrarInformacion = new JButton("Mostrar información");
        btnMostrarInformacion.setFont(new Font("Segoe UI", Font.BOLD, 12));
        btnMostrarInformacion.setBackground(new Color(220, 235, 252));
        btnMostrarInformacion.setForeground(new Color(15, 30, 80));

        panelBotonesVisualizacion.add(btnLimpiarConsola);
        panelBotonesVisualizacion.add(btnMostrarInformacion);
        panelDerecho.add(panelBotonesVisualizacion, BorderLayout.SOUTH);

        // Ensamblar en contenedor
        panelContenedor.add(panelIzquierdo);
        panelContenedor.add(panelDerecho);

        this.add(panelContenedor, BorderLayout.CENTER);
    }

    /**
     * Vinculación de eventos mediante ActionListener.
     */
    private void asignarEventos() {
        // Evento Registrar Curso
        btnRegistrarCurso.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                registrarCurso();
            }
        });

        // Evento Agregar Tarea
        btnAgregarTarea.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                agregarTarea();
            }
        });

        // Evento Mostrar Información
        btnMostrarInformacion.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                mostrarInformacion();
            }
        });

        // Evento Limpiar Pantalla
        btnLimpiarConsola.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                txtAreaVisualizacion.setText("");
            }
        });

        // Evento Convertir a Fahrenheit
        btnConvertirFahrenheit.addActionListener(new ActionListener() {
            @Override
```

```
            public void actionPerformed(ActionEvent e) {
                convertirTemperatura(true);
            }
        });

        // Evento Convertir a Kelvin
        btnConvertirKelvin.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                convertirTemperatura(false);
            }
        });
    }

    /**
     * Valida y registra un nuevo curso.
     */
    private void registrarCurso() {
        String codigo = txtCodigoCurso.getText().trim();
        String nombre = txtNombreCurso.getText().trim();
        String tutor = txtTutorCurso.getText().trim();

        // Validación de campos vacíos
        if (codigo.isEmpty() || nombre.isEmpty() || tutor.isEmpty()) {
            JOptionPane.showMessageDialog(this,
                    "Todos los campos del curso son obligatorios. Por favor, complételos.",
                    "Validación - Campos Vacíos",
                    JOptionPane.WARNING_MESSAGE);
            return;
        }

        // Validación de código duplicado
        for (Curso c : listaCursos) {
            if (c.getCodigo().equalsIgnoreCase(codigo)) {
                JOptionPane.showMessageDialog(this,
                        "Ya existe un curso registrado con el código: " + codigo,
                        "Validación - Curso Duplicado",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }
        }

        // Creación del objeto Curso y almacenamiento
        Curso nuevoCurso = new Curso(codigo, nombre, tutor);
        listaCursos.add(nuevoCurso);
        comboCursos.addItem(nuevoCurso);

        // Limpiar campos
        txtCodigoCurso.setText("");
        txtNombreCurso.setText("");
        txtTutorCurso.setText("");

        JOptionPane.showMessageDialog(this,
                "Curso '" + nombre + "' (Código: " + codigo + ") registrado con éxito.",
                "Registro Exitoso",
                JOptionPane.INFORMATION_MESSAGE);
    }

    /**
     * Valida y asocia una nueva tarea al curso seleccionado.
     */
    private void agregarTarea() {
        // Validación de existencia de cursos
        if (listaCursos.isEmpty() || comboCursos.getSelectedItem() == null) {
            JOptionPane.showMessageDialog(this,
                    "No hay cursos registrados. Debe registrar un curso antes de agregar tareas.",
                    "Validación - Sin Cursos",
                    JOptionPane.WARNING_MESSAGE);
            return;
        }

        Curso cursoSeleccionado = (Curso) comboCursos.getSelectedItem();
        String titulo = txtTituloTarea.getText().trim();
```

```
        String descripcion = txtDescripcionTarea.getText().trim();
        String fecha = txtFechaEntrega.getText().trim();

        // Validación de campos vacíos
        if (titulo.isEmpty() || descripcion.isEmpty() || fecha.isEmpty()) {
            JOptionPane.showMessageDialog(this,
                    "Debe completar todos los datos de la tarea (título, descripción y fecha de entrega).",
                    "Validación - Campos Vacíos",
                    JOptionPane.WARNING_MESSAGE);
            return;
        }

        // Creación del objeto TareaAcademica y asociación al curso
        TareaAcademica nuevaTarea = new TareaAcademica(titulo, descripcion, fecha, cursoSeleccionado);
        cursoSeleccionado.agregarTarea(nuevaTarea);

        // Limpiar campos
        txtTituloTarea.setText("");
        txtDescripcionTarea.setText("");
        txtFechaEntrega.setText("");

        JOptionPane.showMessageDialog(this,
                "Tarea '" + titulo + "' agregada con éxito al curso " + cursoSeleccionado.getNombre() + ".",
                "Tarea Registrada",
                JOptionPane.INFORMATION_MESSAGE);
    }

    /**
     * Despliega la información ordenada de todos los cursos y sus tareas asociadas.
     */
    private void mostrarInformacion() {
        if (listaCursos.isEmpty()) {
            JOptionPane.showMessageDialog(this,
                    "No hay cursos ni tareas registradas para mostrar.",
                    "Información Vacía",
                    JOptionPane.INFORMATION_MESSAGE);
            txtAreaVisualizacion.setText("No hay cursos registrados en el sistema.\n");
            return;
        }

        StringBuilder sb = new StringBuilder();
        sb.append("========================================================================\n");
        sb.append("               REPORTE GENERAL DE CURSOS Y TAREAS ACADÉMICAS          \n");
        sb.append("========================================================================\n\n");

        int totalTareas = 0;
        for (int i = 0; i < listaCursos.size(); i++) {
            Curso c = listaCursos.get(i);
            sb.append(String.format("[%d] CURSO: %s (CÓDIGO: %s)\n", (i + 1), c.getNombre(), c.getCodigo()));
            sb.append(String.format("    Tutor Responsable: %s\n", c.getTutor()));
            sb.append(String.format("    Cantidad de tareas asignadas: %d\n", c.getTareas().size()));

            if (c.getTareas().isEmpty()) {
                sb.append("    -> No tiene tareas asignadas aún.\n");
            } else {
                sb.append("    -> Listado de Tareas:\n");
                for (int j = 0; j < c.getTareas().size(); j++) {
                    TareaAcademica t = c.getTareas().get(j);
                    sb.append(String.format("       %d.%d Título: %s\n", (i + 1), (j + 1), t.getTitulo()));
                    sb.append(String.format("           Descripción: %s\n", t.getDescripcion()));
                    sb.append(String.format("           Fecha Entrega: %s\n", t.getFechaEntrega()));
                }
            }
            sb.append("------------------------------------------------------------------------\n");
            totalTareas += c.getTareas().size();
        }

        sb.append(String.format("Resumen General: Total Cursos: %d | Total Tareas: %d\n", listaCursos.size(),
totalTareas));
        sb.append("========================================================================\n");

        txtAreaVisualizacion.setText(sb.toString());
        txtAreaVisualizacion.setCaretPosition(0);
```

```
    }

    /**
     * Valida y realiza la conversión de temperatura de Celsius a Fahrenheit o Kelvin.
     * @param aFahrenheit true para Fahrenheit, false para Kelvin
     */
    private void convertirTemperatura(boolean aFahrenheit) {
        String textoCelsius = txtCelsius.getText().trim();

        // 1. Validación de campo vacío
        if (textoCelsius.isEmpty()) {
            JOptionPane.showMessageDialog(this,
                    "Por favor, ingrese un valor de temperatura en grados Celsius.",
                    "Validación - Campo Vacío",
                    JOptionPane.WARNING_MESSAGE);
            return;
        }

        // 2. Validación de valor numérico
        double celsius;
        try {
            celsius = Double.parseDouble(textoCelsius);
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this,
                    "El valor ingresado ('" + textoCelsius + "') no es un número válido. Use números enteros o decimales
(ej. 25 o 36.5).",
                    "Validación - Error de Formato",
                    JOptionPane.ERROR_MESSAGE);
            return;
        }

        // 3. Cálculos
        if (aFahrenheit) {
            // Fórmula: F = C * (9/5) + 32
            double fahrenheit = (celsius * 9.0 / 5.0) + 32.0;
            String resultadoStr = df.format(fahrenheit) + "ºF";
            lblResultadoConversion.setText("Resultado: " + resultadoStr);

            JOptionPane.showMessageDialog(this,
                    celsius + " ºC equivale a " + resultadoStr,
                    "Conversión a Fahrenheit",
                    JOptionPane.INFORMATION_MESSAGE);
        } else {
            // Fórmula: K = C + 273.15
            double kelvin = celsius + 273.15;
            String resultadoStr = df.format(kelvin) + " K";
            lblResultadoConversion.setText("Resultado: " + resultadoStr);

            JOptionPane.showMessageDialog(this,
                    celsius + " ºC equivale a " + resultadoStr,
                    "Conversión a Kelvin",
                    JOptionPane.INFORMATION_MESSAGE);
        }
    }
}
```

3.4 Clase de Inicialización: Main.java

```
package tarea4.principal;

import tarea4.vista.VentanaPrincipal;

import javax.swing.SwingUtilities;
import javax.swing.UIManager;

/**
 * Clase de entrada (Entry Point) del programa.
 */
public class Main {
    public static void main(String[] args) {
        // Establecer Look and Feel del sistema para una apariencia moderna
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception e) {
            // Si falla, se usa el Look and Feel por defecto de Java Swing (Metal)
            System.err.println("No se pudo cargar el Look & Feel del sistema: " + e.getMessage());
        }

        // Ejecutar la interfaz gráfica en el hilo de despacho de eventos (EDT)
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                VentanaPrincipal ventana = new VentanaPrincipal();
                ventana.setVisible(true);
            }
        });
    }
}
```

4. Evidencias de Ejecución y Pruebas del Sistema

A continuación se presentan las evidencias fotográficas de la aplicación en funcionamiento real, validando la resolución del caso de uso planteado en la guía de la tarea.

4.1 Estado Inicial de la Aplicación

Al arrancar el software, la ventana se despliega organizada en dos columnas equilibradas: el panel izquierdo contiene los tres módulos de captura y conversión, mientras que el panel derecho expone el área de visualización con barras de desplazamiento.

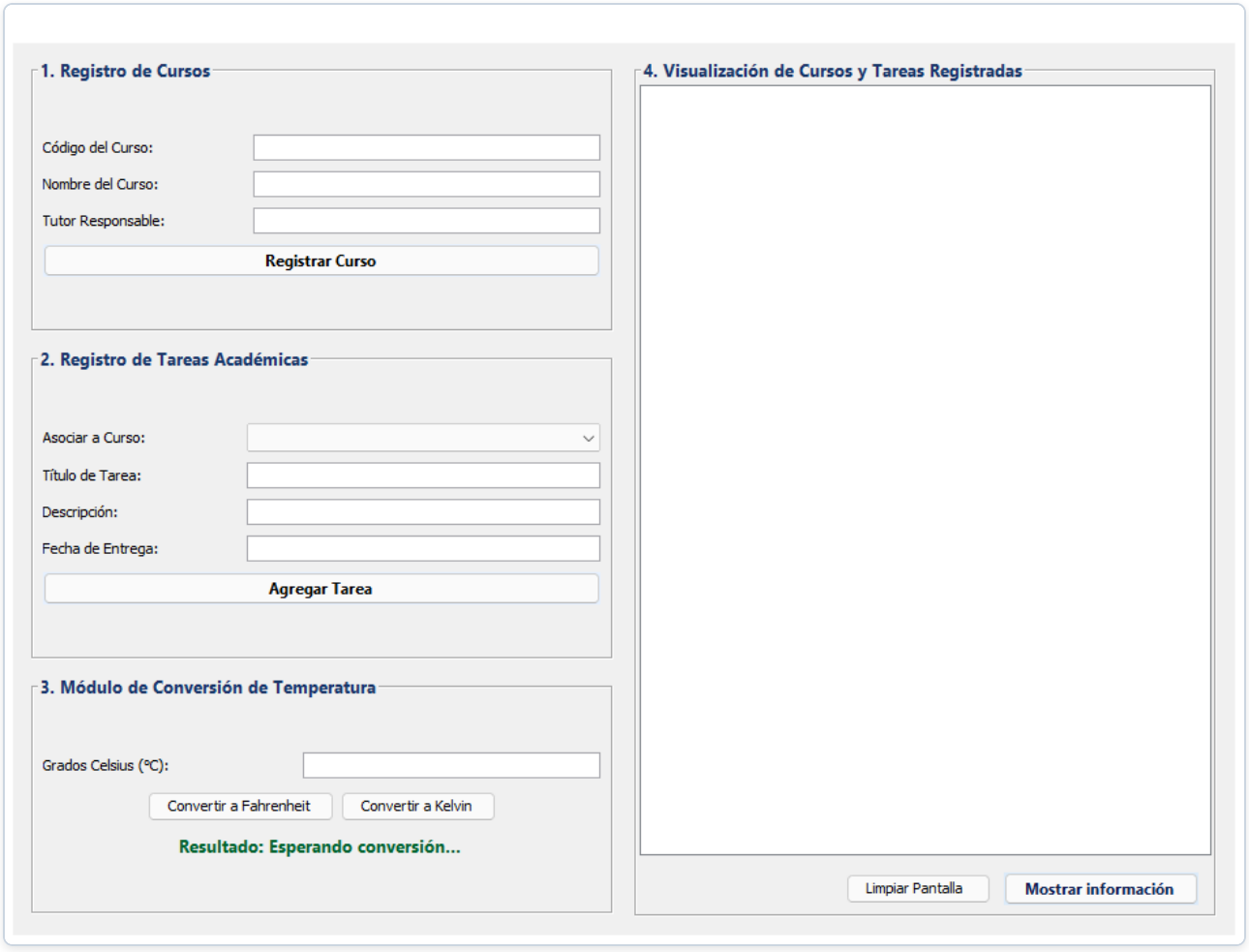


Figura 1: Interfaz gráfica inicial programada 100% por código en Java Swing.

4.2 Registro de Cursos, Tareas y Visualización Jerárquica

Se registra el curso **IPC1 (Código: 0770)** con su tutor responsable. Seguidamente se asocia la tarea "*Laboratorio POO*" y una segunda tarea. Al pulsar **"Mostrar información"**, el sistema despliega el reporte estructurado en el `JTextArea`. Asimismo, se ingresan **25 °C** y se presiona **"Convertir a Fahrenheit"**, obteniendo **77 °F**.

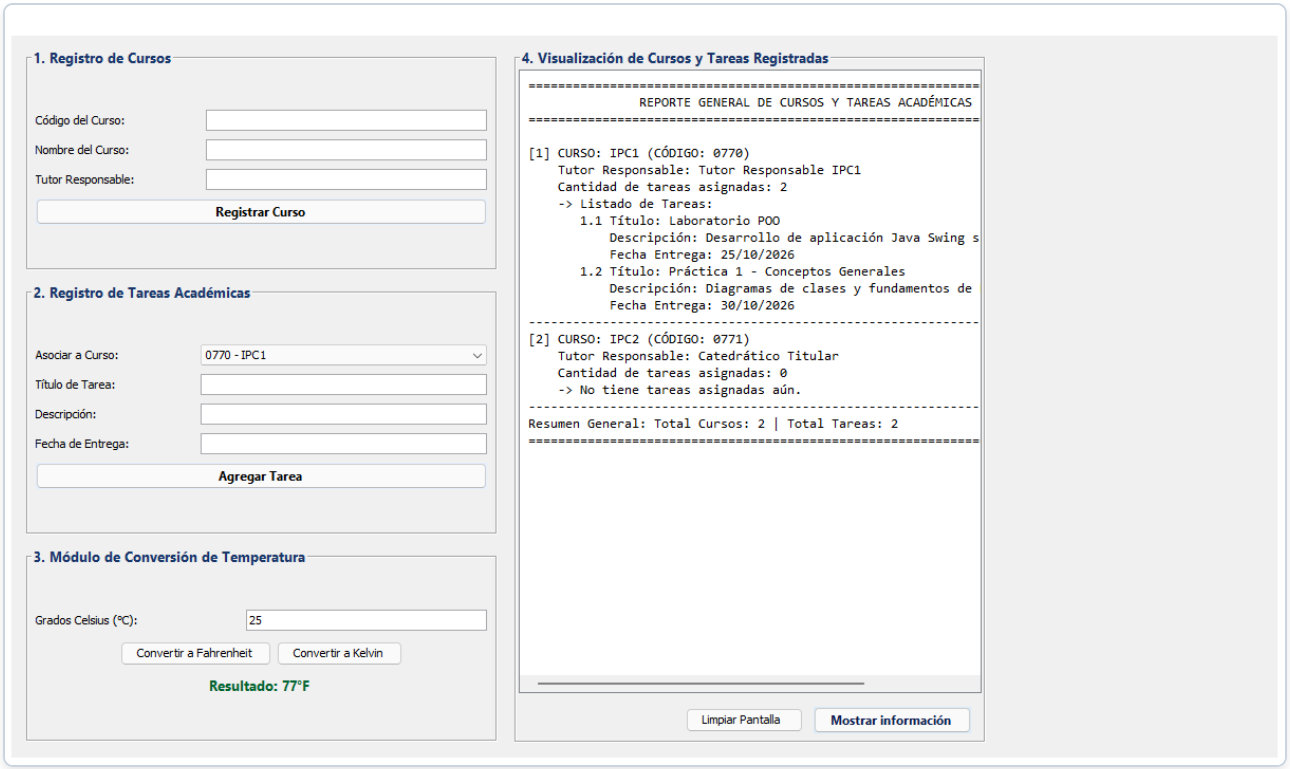


Figura 2: Despliegue de cursos y tareas asociadas en el JTextArea y resultado de conversión a Fahrenheit (25 °C = 77 °F).

4.3 Conversión de Temperatura a Escala Kelvin

Al mantener la temperatura de **25 °C** y pulsar el botón **"Convertir a Kelvin"**, el algoritmo ejecuta la adición con la constante 273.15, actualizando la etiqueta de resultado y reflejando exactamente **298.15 K** como se solicita en el ejemplo de la guía.

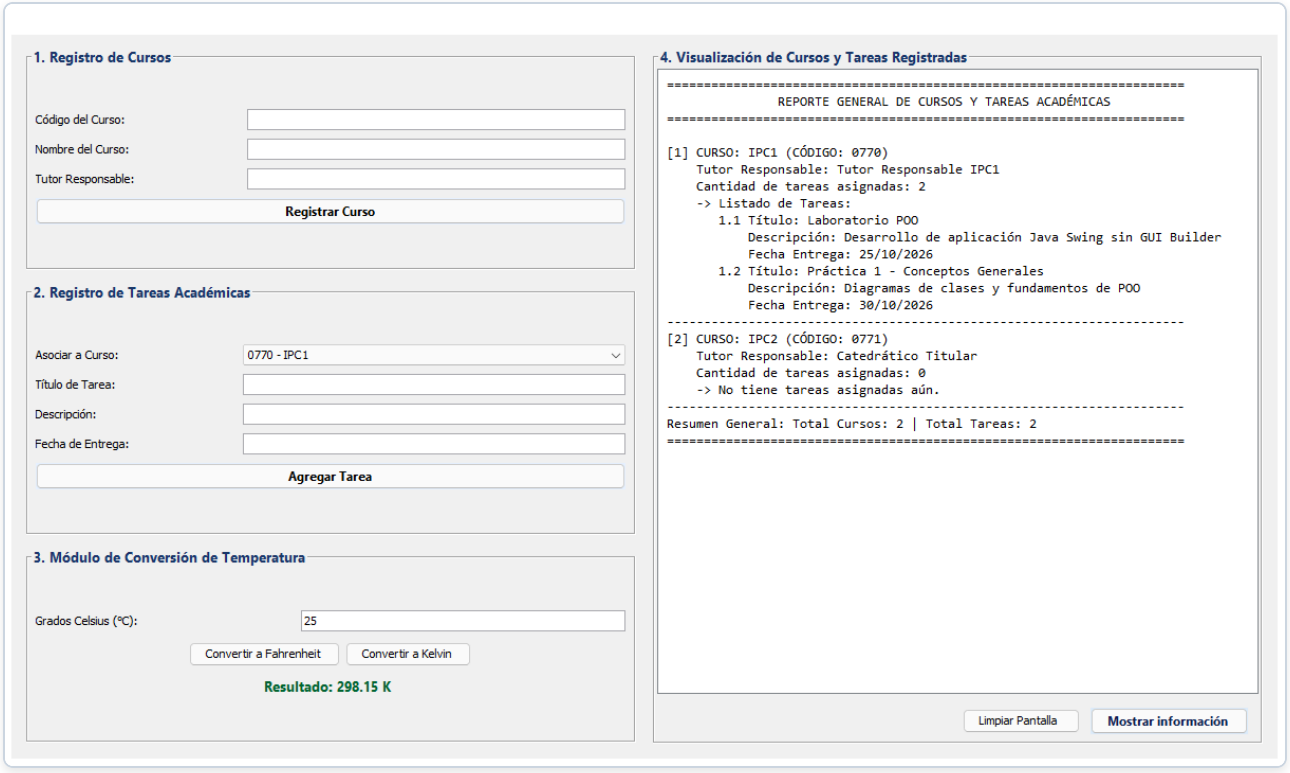


Figura 3: Conversión térmica precisa a escala Kelvin mostrando 298.15 K.

4.4 Validación Preventiva de Datos mediante Diálogos JOptionPane

Para verificar el cumplimiento de los requerimientos de validación de datos, se realizaron pruebas ingresando texto inválido en el conversor y registrando cursos satisfactoriamente:

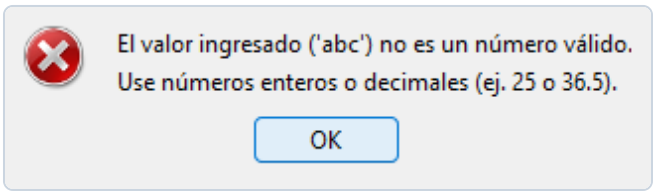


Figura 4: Diálogo modal de error ante entradas alfanuméricas inválidas.

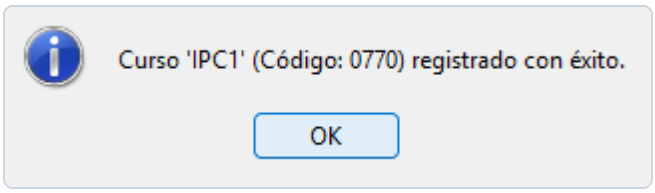


Figura 5: Diálogo modal confirmando el registro exitoso del curso.

5. Conclusiones

1. **Separación de Responsabilidades y Encapsulamiento:** La aplicación rigurosa de los pilares de la POO permitió aislar el modelo del dominio académico (`Curso` y `TareaAcademica`) de los componentes de la interfaz de usuario, garantizando un código fácil de depurar, modular y altamente mantenible.
2. **Dominio de Interfaces Gráficas por Código Puro:** Prescindir de herramientas de arrastrar y soltar (GUI Builders) promueve una comprensión profunda de la jerarquía de contenedores de Swing, el comportamiento responsivo de

los administradores de diseño (`BorderLayout` , `GridBagLayout` , `BoxLayout`) y la manipulación directa de propiedades visuales.

3. **Seguridad y Robustez en Tiempo de Ejecución:** La validación proactiva de campos vacíos y el encierro de conversiones de cadenas a tipos numéricos dentro de bloques `try-catch(NumberFormatException)` respaldados por diálogos `JOptionPane` blindan a la aplicación frente a anomalías o fallos inesperados generados por el usuario.

6. Referencias Bibliográficas

- Oracle Corporation. (2024). *The Java™ Tutorials: Creating a GUI With JFC/Swing*. Oracle Documentation. Recuperado de: https://docs.oracle.com/javase/tutorial/uiswing/
- Deitel, P. J., & Deitel, H. M. (2018). *Java: How to Program (Early Objects)* (11.ª ed.). Pearson Education.
- W3Schools. (2024). *Java OOP (Object-Oriented Programming) Tutorial*. Recuperado de: https://www.w3schools.com/java/
- Eckel, B. (2006). *Thinking in Java* (4.ª ed.). Prentice Hall.